¿Qué pasaría si...?

Experimentación con JMeter sobre la API de Tiquetmania: carga, estrés y concurrencia

Pablo Guzmán · Automatización de Pruebas 2026-2

github.com/pabloandressoftware/tiquetmania

Cómo se hicieron las pruebas

1 Un solo plan

Experimentacion.jmx se configura con propiedades: usuarios, subida, duración, pausas y flujo.

2 18 escenarios

Cada pregunta es una ejecución distinta del mismo plan, en modo no-GUI, con la API recién iniciada.

3 Medición propia

CPU y memoria cada segundo del equipo, de JMeter y de la API, con un muestreador en Python.

4 Misma máquina

JMeter y la API comparten computador: es la principal limitación de todas las pruebas.

Lo más importante en cuatro datos

~2 300

peticiones por segundo: donde se satura la API

0 %

errores con carga normal, incluso con 500 usuarios

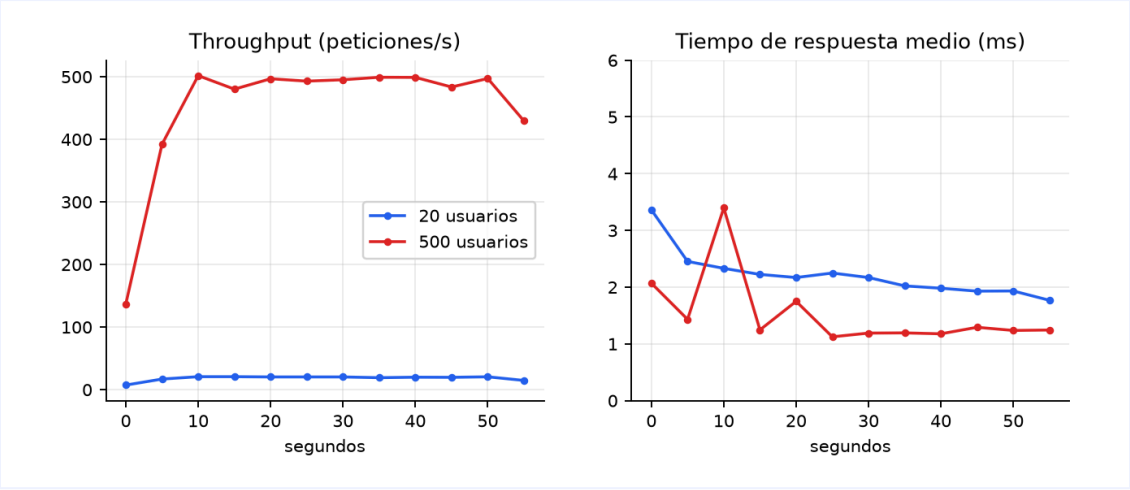
16,6 y 41 %

de errores cuando 500 usuarios llegan al mismo tiempo (2 corridas)

36 420

totales validados bajo carga: siempre precio × cantidad

P1 · De 20 a 500 usuarios



Mi respuesta

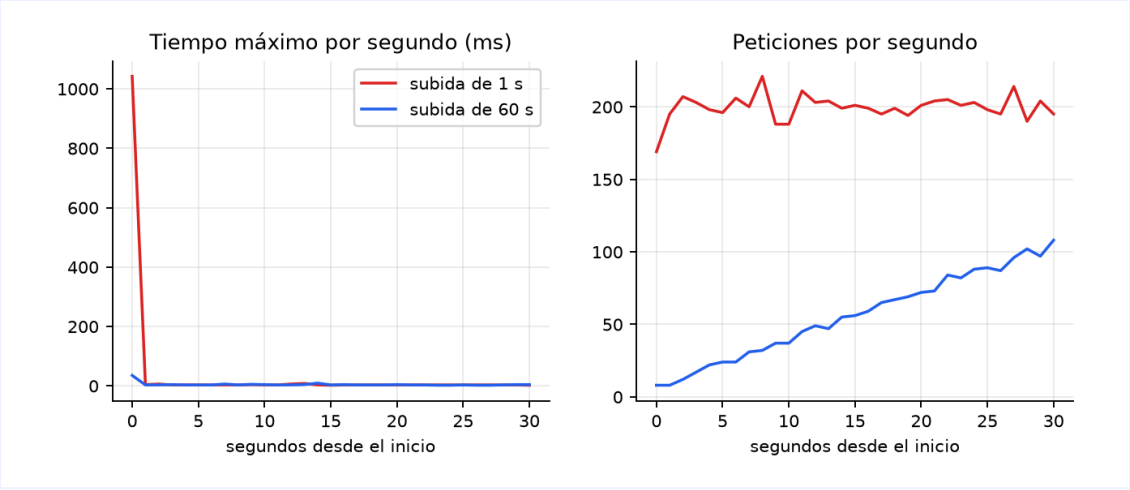
- Cambia el throughput; el tiempo de respuesta no.
- 25 veces más usuarios, casi 25 veces más peticiones por segundo.
- 500 usuarios con pausas no presionan a la API.

~20 → ~495
peticiones por segundo

2,2 → 1,5
ms de tiempo medio

0 %
errores

P2 · Ramp-up de 1 s a 60 s



1 042 ms

máximo en 5 s, subida de 1 s

35 ms

máximo en 5 s, subida de 60 s

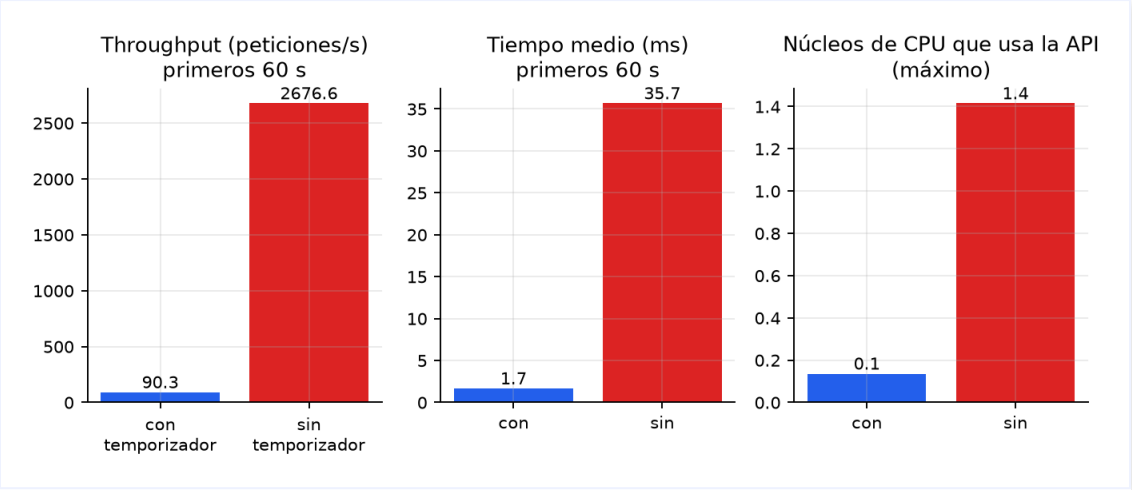
1,5 ms

media, en los dos

Mi respuesta

- Sí cambia el pico inicial, pero es de muy pocas peticiones.
- Con 60 s la carga sube en rampa y el pico desaparece.
- Ya en marcha, los dos escenarios se comportan igual.

P3 · Quitar los temporizadores



Mi respuesta

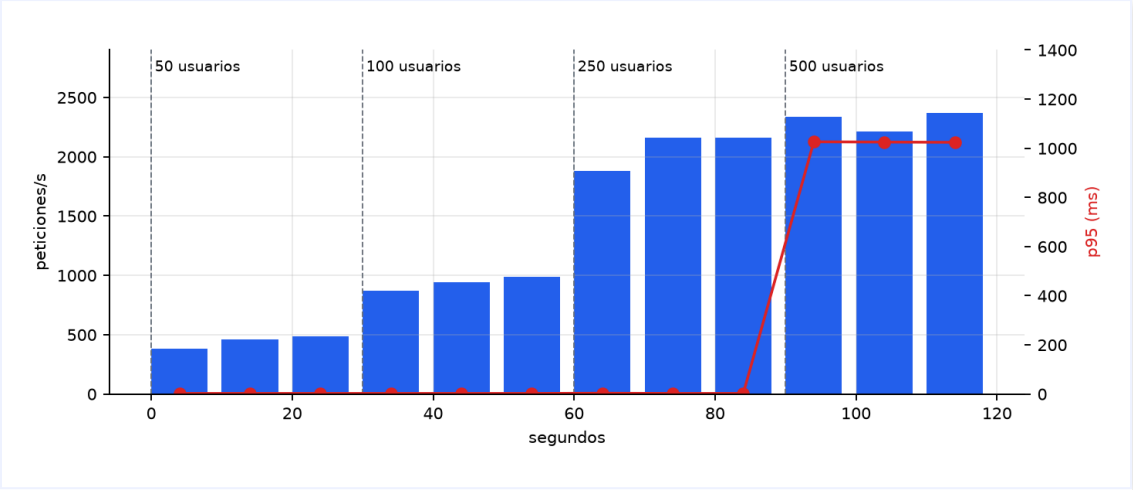
- El throughput se multiplica por unas 30 veces.
- Sin pausas ya no simulas personas: mides el límite de la API.
- La prueba de 60 s se alargó a 124 s por respuestas lentas.

~90 → ~2 680
peticiones por segundo

×30
más throughput

1 052 ms
p99 sin pausas

P4 · Escalones de 50 a 500 usuarios



~480 → ~985

50 → 100 usuarios

≈ 2 300

con 250 y 500 usuarios: se aplana

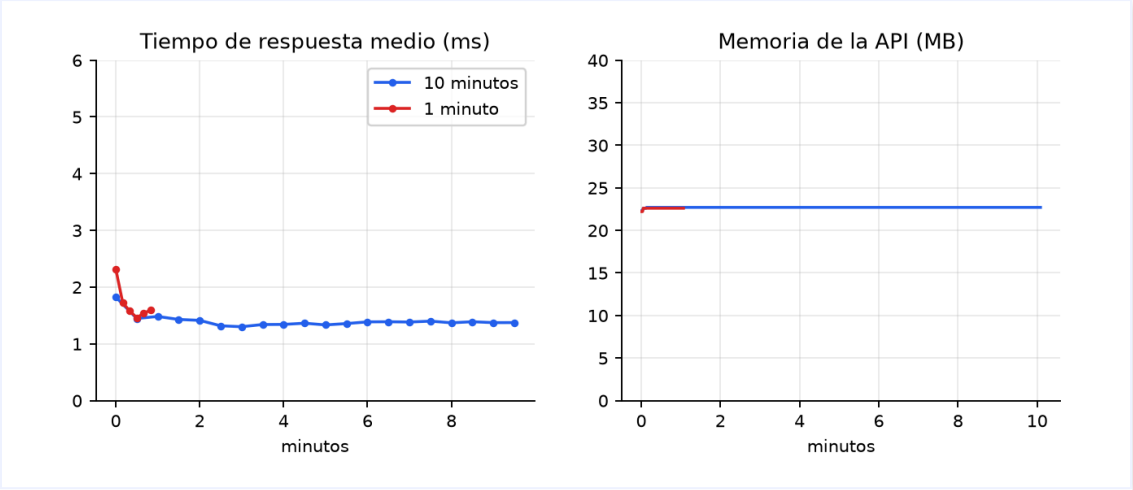
2 → 1 025

ms de p95 con 500 usuarios

Mi respuesta

- Deja de crecer entre 250 y 500 usuarios, cerca de 2 300 peticiones por segundo.
- Con 500 usuarios el p95 pasa de 2 ms a más de 1 s.
- Sin errores: la saturación se ve como demora.

P5 · 10 minutos en vez de 1



Mi respuesta

- Se mantiene: no sube con el tiempo.
- La memoria de la API tampoco crece.
- Con esta carga, 10 minutos no muestran ningún problema.

1,7 vs 1,4

ms de media en 1 y 10 minutos

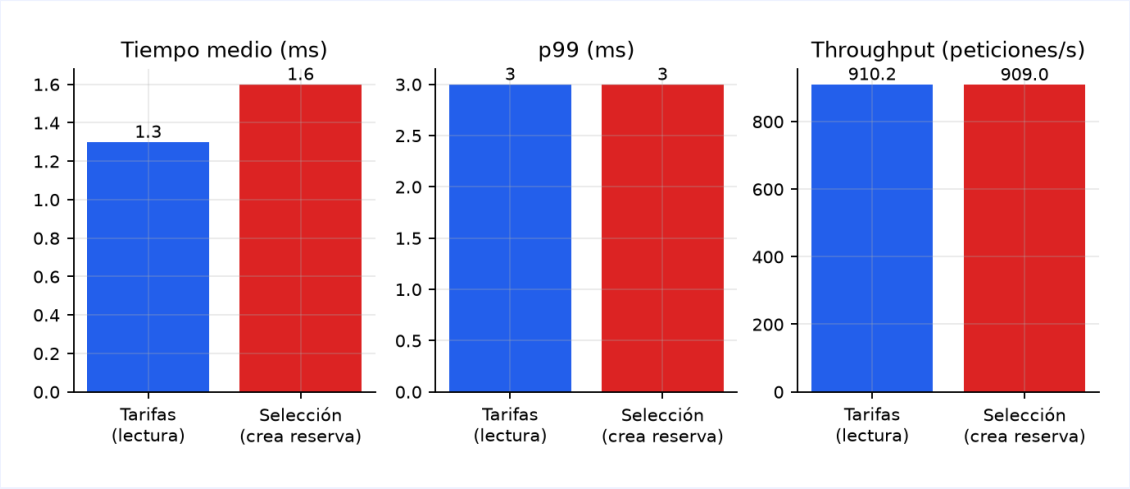
22,3 → 22,7

MB de memoria de la API

0 %

errores

P6 · Tarifas vs selección



Mi respuesta

- Cuesta más /api/seleccion, pero por poco: 0,3 ms.
- Valida, genera un identificador y guarda la reserva en memoria.
- Si nadie paga, la memoria de la API se duplica.

1,3 vs 1,6

ms: tarifas vs selección

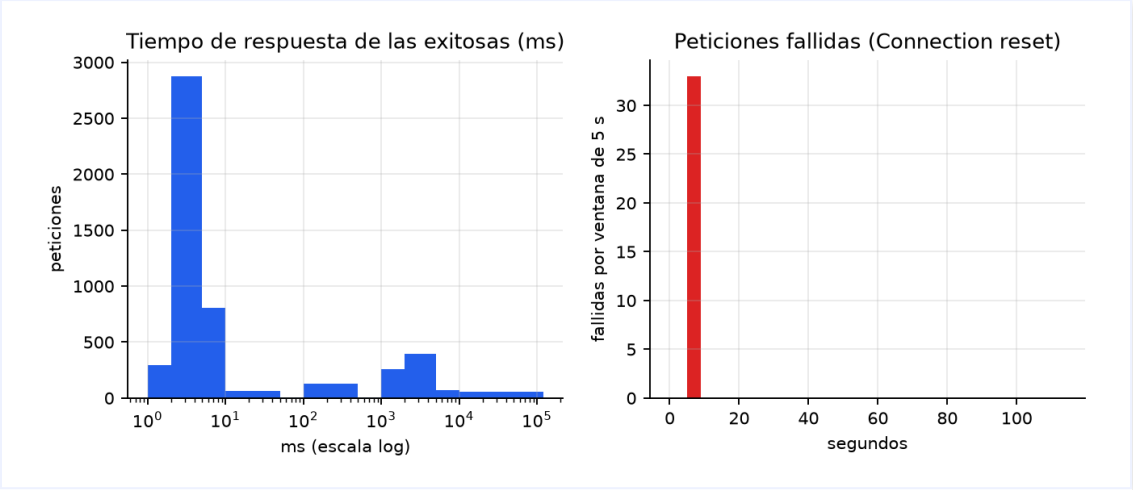
+23 %

costo de selección

22 → 44

MB de memoria de la API

P7 · 500 reservas al mismo tiempo



33 de 5 000

fallaron en la 1.ª corrida

0 de 5 000

fallaron en la 2.ª corrida

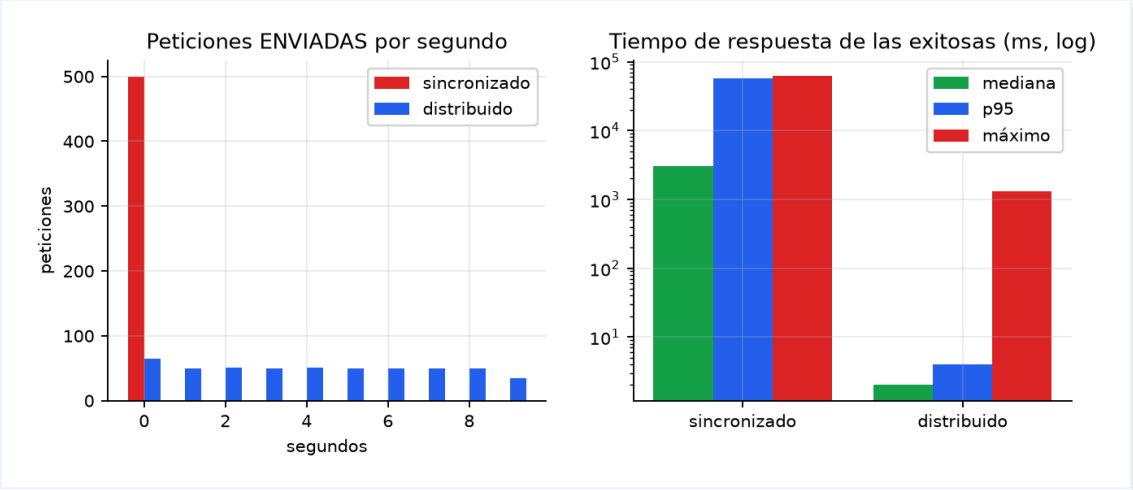
3,7 → 2,5 s

p95, 1.ª y 2.ª corrida

Mi respuesta

- A veces no: la 1.ª vez fallaron 33 con “Connection reset”; la 2.ª vez, ninguna.
- Lo que sí se repite: tiempos de varios segundos.
- Las reservas creadas fueron correctas.

P8 · Sincronizado vs distribuido



Mi respuesta

- El sincronizado dispara las 500 peticiones en el mismo segundo.
- Repartidas en 10 s, ninguna falla (en las dos corridas).
- El % de error del sincronizado cambia entre corridas.

16,6 / 41 %

error sincronizado (1.ª / 2.ª)

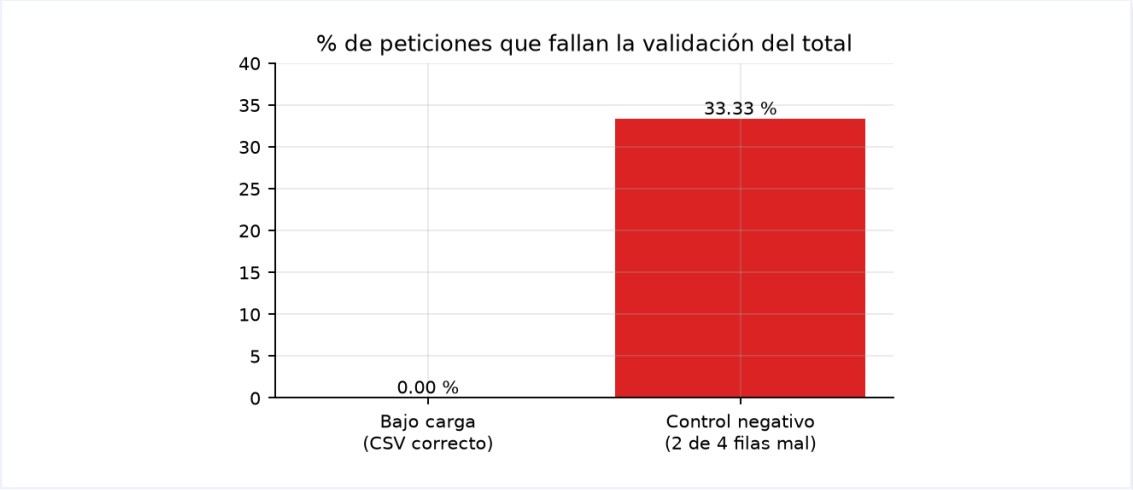
0 %

error, distribuido

3 s vs 2 ms

mediana de las exitosas

P9 · ¿El total cobrado coincide?



Mi respuesta

- El total siempre fue precio × cantidad, también bajo carga.
- Validación: columna total_esperado en el CSV y JSON Assertion.
- El control negativo prueba que la validación sí detecta errores.

36 420

totales validados

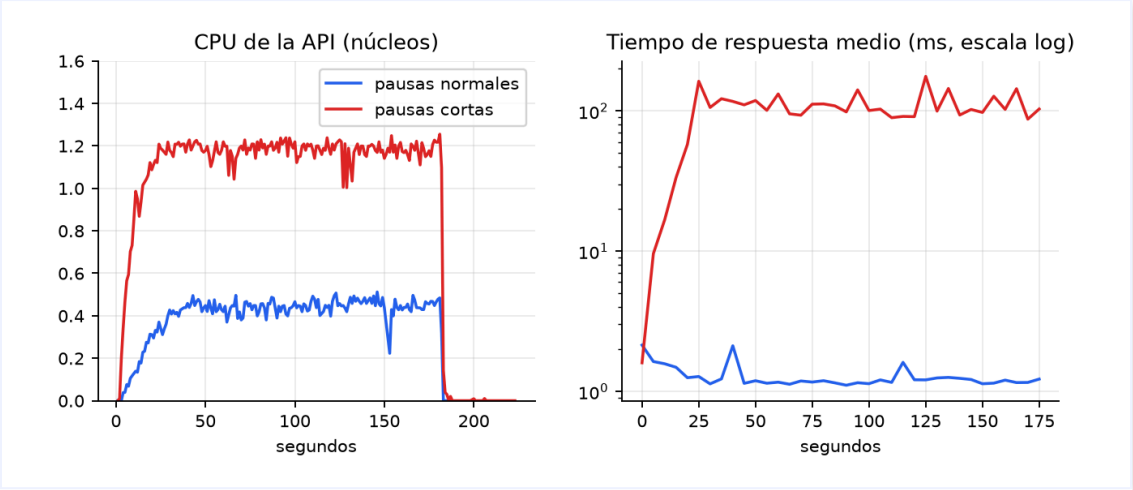
0

diferencias bajo carga

40 de 120

fallan en el control negativo

P10 · CPU y memoria con 500 usuarios



Mi respuesta

- Con pausas normales nada se satura.
- Con pausas cortas la API pasa de 0,5 a 1,25 núcleos y se estanca.
- El CPU del equipo no pasó del 45 %: el límite es el proceso de la API.

40 %

CPU máx. del equipo

0,5 → 1,25

núcleos que usa la API

1,2 → 104

ms de tiempo medio

Repetí cada escenario dos veces

16 de 17

escenarios dan la misma conclusión en la segunda corrida

0 %

errores con carga normal, en las dos corridas

33 → 0

peticiones fallidas en la P7: ese error no se repitió

16,6 → 41 %

errores del sincronizado en la P8: el tamaño varía

Conclusiones

- Con pausas normales la API aguanta 500 usuarios: sin errores y sin cambios en el tiempo de respuesta.
- Se satura cerca de 2 300 peticiones por segundo; después los tiempos se disparan, sin errores.
- El límite es el proceso de la API, no el equipo completo.
- Lo más dañino es la llegada simultánea de conexiones: 500 al mismo tiempo producen errores, pero su tamaño varía entre corridas.
- La lógica de negocio es correcta bajo carga: el total siempre fue precio × cantidad.

Limitaciones: dos corridas por escenario (la de 10 min, una), JMeter y la API en el mismo equipo, sin PerfMon ni Ultimate Thread Group.